Домашнее задание: автоматизация тестирования REST и GraphQL API

Техническая информация

Параметр	Значение
Дисциплина	Тестирование ПО · ИТМО · осень 2026
Занятие	4 — «Тестирование API: REST, GraphQL и RestAssured»
Проект	`самостоятельный Maven/Python/go -проект`
Стек	Java - JUnit 5 / python - pytest / go - testing
Трудоёмкость	3–5 часов
Количество попыток	1
Оценивание	до 12 баллов
Проверяющий	преподаватель
Формат оценивания	по критериям
Дедлайн	19.10
Разбор задания	20.10

* При схеме аттестации **60 баллов за 5 домашних заданий + 40 баллов за итоговый проект**.

Цель задания

Научиться строить связанные API-автотесты: формировать REST-запросы, извлекать данные из ответов и передавать их в следующие шаги, работать с Bearer-аутентификацией и проверять GraphQL query, mutation и subscription.

Ваш результат

В результате выполнения задания вы:

- создадите `Maven/Python/go` с API-автотестами;
- реализуете связанный REST-сценарий `POST → id → PATCH → GET`;
- получите `access_token` и автоматически передадите его в защищённый запрос;
- автоматизируете GraphQL `query` и `mutation` и выполните `subscription` в Postman;
- добавите содержательные проверки ответов, негативный сценарий, логирование и README с командой запуска.

Материалы для выполнения

Для работы используйте:

1. REST API: `https://playground.nileslabs.com/api/v1`.
2. GraphQL endpoint: `https://graphql.postman-echo.com/graphql`.
3. Postman collection, использованную на семинаре.
4. Тестовые данные для login — те же, что использовались на семинаре: `Bret / password123`.

Публичные сервисы используются как учебные стенды. Не полагайтесь на заранее существующее изменяемое состояние: необходимые id и токены получайте в ходе собственного тестового сценария.

Подготовка(для варианта на java)

1. Создайте Maven-проект или используйте подготовленный преподавателем шаблон.
2. Добавьте зависимости JUnit 5 и RestAssured.
3. Убедитесь, что из корня проекта выполняется команда `mvn test`.
4. Импортируйте учебную Postman collection и проверьте доступность REST и GraphQL endpoint'ов.

Задача

Ваша задача — реализовать три обязательных блока: связанный REST-сценарий, авторизацию между запросами и GraphQL-проверки.
Ключевой принцип задания: **request → response → extract → следующий request**.
Не ограничивайтесь проверкой HTTP-кода. Тест должен подтверждать, что после запроса система вернула ожидаемые данные или перешла в ожидаемое состояние.

Что нужно сделать

1. Подготовьте структуру проекта

Проект должен содержать как минимум:

- `pom.xml` с необходимыми зависимостями;
- `src/test/java` с тестами;
- `README.md` с описанием API, реализованных сценариев и командой запуска.

Повторяющиеся настройки (base URI, Content-Type, логирование) желательно вынести в RequestSpecification или общий helper.

2. Реализуйте связанный REST-сценарий

Используйте Playground API и выполните последовательность:

1. `POST /posts` — создайте новый пост.
2. Проверьте успешный status code и извлеките `id` из JSON-ответа.
3. `PATCH /posts/{id}` — измените `title` созданного поста.
4. `GET /posts/{id}` — получите тот же пост.
5. Проверьте, что `id` совпадает, `title` изменился, а хотя бы одно поле, которое PATCH не менял, сохранило исходное значение.

Пример ключевой части сценария:

```
int id = given().body(post).when().post("/posts")
    .then().statusCode(201).extract().path("id");
```

Динамически полученный id должен использоваться в следующих запросах. Хардкодить id созданного ресурса нельзя.

3. Реализуйте авторизацию между запросами

1. `POST /auth/login` — отправьте login/password.
2. Проверьте HTTP 200 и наличие непустого строкового поля `access_token`.
3. Извлеките token из ответа.
4. `GET /auth/me` — передайте token как `Authorization: Bearer <token>`.
5. Проверьте успешный ответ и идентифицирующие поля пользователя.
6. Добавьте негативный сценарий: отсутствующий или заведомо неверный token должен приводить к отказу в доступе.

Готовый `access_token` нельзя хранить в коде. Тест должен получать его во время выполнения.

4. Реализуйте GraphQL-проверки

Используйте endpoint:
`https://graphql.postman-echo.com/graphql`
Query:

- отправьте простой `query`;
- проверьте HTTP 200, наличие ожидаемых данных внутри `data` и отсутствие `errors`.

Mutation:

- выполните `CreatePerson` или эквивалентную mutation;
- проверьте возвращённые `id`, `name` и `age`;
- отдельно проверьте отсутствие `errors`.

Subscription:

- в Postman выполните `subscription Greetings { greetings }`;
- используйте потоковый ответ SSE с заголовком `Accept: text/event-stream`;
- убедитесь, что клиент получает события от сервера.

Автоматизировать subscription в RestAssured не требуется: для этого блока достаточно запроса в экспортируемой Postman collection. Query и mutation должны быть реализованы в Java-тестах.

5. Добавьте диагностику и обеспечьте читаемость тестов

- Добавьте логирование запроса и ответа при падении validation.
- Используйте понятные имена тестов и методов.
- Не дублируйте base URI, общие headers и одинаковую конфигурацию во всех тестах.
- Не заставляйте отдельные `@Test` зависеть от порядка запуска друг друга.
- Связанные шаги одного бизнес-сценария допустимо выполнять внутри одного теста или через явно вызываемые helper-методы/setup.

Важные ограничения

Не хардкодьте динамические `id` и `access_token`.

Не считайте один `statusCode(200)` достаточной проверкой успешности сценария.

Для GraphQL проверяйте и транспортный уровень, и содержимое ответа: `data` и `errors`.

Не используйте ручной порядок запуска тестов как способ передавать данные между разными тест-кейсами.

Не добавляйте `Thread.sleep()` без необходимости.

README должен позволять запустить проект без IDE и без дополнительных устных инструкций.

Критерии проверки

Задание оценивается по следующим критериям. Максимум по основным критериям — 12 баллов.

Критерий	2 балла	1 балл	0 баллов
REST-сценарий	Полная цепочка POST → extract id → PATCH → GET; проверены status, id, изменённые и неизменённые поля.	Сценарий реализован частично: отсутствует один шаг или часть содержательных проверок.	Связанный REST-сценарий отсутствует или id захардкожен.
Авторизация	Token получается через /auth/login, автоматически передаётся в /auth/me; есть позитивный и негативный сценарии.	Авторизация работает, но отсутствует негативная проверка или передача token реализована не полностью.	Авторизация не реализована либо используется заранее захардкоженный access_token.
GraphQL	Есть query и mutation в RestAssured, subscription в Postman; проверяются data, значения полей и отсутствие errors.	Реализована только часть операций или проверки data/errors неполные.	GraphQL-блок отсутствует или фактически не проверяет результат операции.
Полнота проверок	Проверяется не только HTTP status, но и данные/состояние; есть требуемый негативный сценарий и диагностичное логирование.	Основные проверки есть, но часть ограничивается status code или диагностика неполная.	Содержательные assertions отсутствуют или тесты не позволяют определить причину ошибки.
Качество проекта	Проект запускается командой mvn test; есть README; повторяющиеся настройки переиспользуются; динамические данные не хардкодятся; тесты читаемы.	Проект запускается, но часть требований к структуре, README или переиспользованию настроек не выполнена.	Проект не запускается стандартной командой или требует ручной доработки перед запуском.

Критерий	2 балла	1 балл	0 баллов
Итого			**10 баллов**

Дополнительные баллы (+2): **собственный негативный или параметризованный сценарий**, которого не было в обязательной части, с понятным ожидаемым результатом и содержательными проверками.

Что и как сдавать

Отправьте:

1. Maven-проект с `pom.xml` и исходниками тестов в `src/test/java`.
2. `README.md` с описанием используемых API, сценариев и командой запуска.

Формат сдачи: **архив или ссылка на ветку в вашем репозитории**.
Дедлайн: 19.**10**

Команды

Полный прогон тестов:

```
mvn test
```

Команда должна выполняться из корня проекта без предварительного запуска тестов вручную в IDE.